

École Supérieure Privée de Management de Tunis
Esprit School of Business

RAPPORT DE STAGE

En vue de l'obtention du diplôme de
Licence en Business Computing
Parcours **Business Intelligence**

*« Conception et développement d'un système intelligent de gestion des rôles
et de pointage des employés avec tableau de bord décisionnel »*

Du 16/02/2026 au 16/06/2026



Réalisé par :
Oussema Korbosli

Soutenu le :

Maître de stage :
Melek Aziz Elmoula

Encadrant académique :
Heni Abidi

Année universitaire 2025/2026

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Je remercie chaleureusement mon encadrant académique, **Heni Abidi**, pour ses conseils avisés, sa disponibilité et son suivi rigoureux tout au long de ce projet.

Mes remerciements s'adressent également à mon maître de stage, **Melek Aziz Elmoula**, Data Scientist au sein de la **BTK – Banque Tuniso-Koweïtienne**, pour la confiance accordée, l'accompagnement et le partage de l'expérience métier.

Je tiens aussi à remercier l'ensemble du corps enseignant et administratif de l'**Esprit School of Business** pour la qualité de la formation dispensée durant mon cursus en Licence Business Computing, parcours Business Intelligence.

Enfin, j'exprime ma reconnaissance à ma famille et à mes amis pour leur soutien constant et leurs encouragements.

Table des matières

Remerciements	i
Liste des acronymes	vi
Introduction générale	1
1 Cadre général du projet	2
1.1 Présentation de l'organisme d'accueil	2
1.1.1 Aperçu historique	2
1.1.2 Structure et évolution du capital	3
1.1.3 Activités et services	4
1.2 Cadre et contexte du stage	4
1.3 Étude de l'existant	4
1.3.1 Description de l'existant	4
1.3.2 Critique de l'existant	4
1.4 Problématique	4
1.5 Solution proposée	5
1.6 Méthodologie de travail	5
2 Analyse et spécification des besoins	7
2.1 Identification des acteurs	7
2.2 Besoins fonctionnels	7
2.3 Besoins non fonctionnels	8
2.4 Diagramme de cas d'utilisation global	8
2.5 Description de quelques cas d'utilisation	8
2.5.1 Cas « S'authentifier »	8
2.5.2 Cas « Soumettre / valider une demande »	10
3 Conception	11
3.1 Architecture générale	11
3.2 Architecture logique	11
3.3 Architecture physique	12
3.4 Conception détaillée	12
3.4.1 Diagramme de classes	12
3.4.2 Diagramme de séquence : authentification	14
3.4.3 Diagramme de séquence : validation d'une demande	14
3.4.4 Diagramme d'états-transitions : cycle de vie d'une demande	15
3.5 Conception de la base de données	16

3.6	Conception du volet décisionnel	16
4	Réalisation	18
4.1	Environnement de travail	18
4.1.1	Environnement matériel	18
4.1.2	Environnement logiciel	18
4.2	Choix technologiques	19
4.3	Architecture de déploiement	19
4.4	Présentation des interfaces	19
4.4.1	Page d'authentification	19
4.4.2	Tableau de bord	19
4.4.3	Modules de gestion et de demandes	20
4.5	Réalisation du volet décisionnel	20
4.6	Module de pointage	21
4.7	Sécurité et gestion des rôles	21
	Conclusion générale et perspectives	22
	Nétographie	23
A	Annexes	24
A.1	Scripts SQL d'initialisation	24
A.2	Captures complémentaires	24
A.3	Dépôt du code source	24

Table des figures

1.1	Logo de la BTK – Banque Tuniso-Koweïtienne	2
1.2	Planification du projet (diagramme de Gantt des sprints)	5
2.1	Diagramme de cas d'utilisation global	9
3.1	Architecture logique en couches de l'application	12
3.2	Architecture physique (diagramme de déploiement 3-tiers)	13
3.3	Diagramme de classes du modèle métier	13
3.4	Diagramme de séquence – Authentification	14
3.5	Diagramme de séquence – Validation d'une demande	15
3.6	Diagramme d'états-transitions – Cycle de vie d'une demande	15
3.7	Schéma relationnel de la base de données	16
3.8	Modèle décisionnel en étoile (vues V_BI_*)	17
4.1	Interface d'authentification	19
4.2	Tableau de bord de l'application	20
4.3	Modules de gestion et de validation des demandes	20
4.4	Rapport Power BI – analyse des effectifs et des objectifs	20
4.5	Interface du module de pointage	21

Liste des tableaux

1.1	Fiche signalétique de la BTK	3
1.2	Évolution de la structure du capital de la BTK	3
1.3	Découpage du projet en sprints Scrum	5
2.1	Identification des acteurs	7
2.2	Besoins non fonctionnels	8
3.1	Indicateurs de performance (KPI) du volet décisionnel	17
4.1	Environnement logiciel	18

Liste des acronymes

Acronyme	Signification
BTK	Banque Tuniso-Koweïtienne
BI	Business Intelligence (informatique décisionnelle)
KPI	Key Performance Indicator (indicateur clé de performance)
PFE	Projet de Fin d'Études
UML	Unified Modeling Language
JPA	Jakarta Persistence API
EJB	Enterprise Java Beans
JSP	Jakarta Server Pages
JSTL	Jakarta Standard Tag Library
JTA	Jakarta Transaction API
JDBC	Java Database Connectivity
MVC	Modèle–Vue–Contrôleur
SGBD	Système de Gestion de Base de Données
WAR	Web Application Archive
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol

Introduction générale

La transformation numérique du secteur bancaire impose aux établissements de disposer de systèmes d'information capables, d'une part, de centraliser la gestion de leur réseau d'agences et, d'autre part, d'exploiter leurs données pour piloter l'activité. La maîtrise de l'information — effectifs, portefeuille clients, objectifs commerciaux — constitue aujourd'hui un levier déterminant de performance et d'aide à la décision.

C'est dans ce contexte que s'inscrit ce projet de fin d'études, réalisé au sein de la **BTK – Banque Tuniso-Koweïtienne** en vue de l'obtention de la Licence en Business Computing, parcours Business Intelligence, à l'Esprit School of Business. Le projet consiste à concevoir et réaliser une application web de gestion des agences bancaires, adossée à un volet décisionnel destiné à l'aide à la prise de décision.

L'application couvre la gestion des agences, des employés, des clients et des objectifs commerciaux, le **pointage (présence) des employés**, ainsi qu'un ensemble de circuits de validation (workflows) encadrant la création et la modification des données sensibles. À ces fonctionnalités transactionnelles s'ajoute un volet décisionnel : des vues analytiques exposées à Microsoft Power BI et un tableau de bord embarqué offrant aux responsables une vision synthétique de l'activité.

Ce rapport est organisé en quatre chapitres. Le **premier** présente le cadre général du projet : l'organisme d'accueil, l'étude de l'existant, la problématique, la solution proposée et la méthodologie de travail. Le **deuxième** est consacré à l'analyse et à la spécification des besoins. Le **troisième** détaille la conception de l'application et de son volet décisionnel, en s'appuyant sur les architectures logique et physique et sur les diagrammes UML. Le **quatrième** décrit la réalisation, l'environnement et les technologies mises en œuvre. Une conclusion générale dresse le bilan du travail et ouvre des perspectives d'évolution.

Chapitre 1

Cadre général du projet

Introduction

Ce premier chapitre situe le projet dans son contexte. Il présente l'organisme d'accueil, analyse la situation existante, dégage la problématique, expose la solution retenue et précise la méthodologie de travail adoptée durant le stage.

1.1 Présentation de l'organisme d'accueil

La **Banque Tuniso-Koweïtienne** (BTK Bank) est une banque universelle de droit tunisien, créée le 25 février 1981 dans le cadre d'une convention de coopération bilatérale entre la Tunisie et le Koweït. Constituée en société anonyme et régie par la loi bancaire n° 2016-48, elle dispose d'un capital social de 200 millions de dinars et a son siège social à Tunis. À travers un réseau d'agences couvrant l'ensemble du territoire tunisien, la BTK accompagne aussi bien les particuliers que les entreprises au moyen d'une offre complète de produits de financement, d'épargne et de services internationaux.



FIGURE 1.1 – Logo de la BTK – Banque Tuniso-Koweïtienne

La figure 1.1 présente le logo de l'institution. Le tableau 1.1 en synthétise la fiche signalétique.

1.1.1 Aperçu historique

Issue de la convention bilatérale signée en 1980 entre la Tunisie et le Koweït, la BTK voit le jour en 1981 avec une vocation initiale de financement des projets d'investissement à forte valeur ajoutée. Elle obtient en 2004 son agrément de banque universelle, élargissant son activité à la banque de détail, au financement des entreprises et aux opérations de marché. Son actionnariat connaît ensuite plusieurs évolutions : l'entrée du groupe français BPCE en 2008, le doublement de son capital en 2017, puis l'acquisition de la participation de référence par le Groupe Elloumi

TABLE 1.1 – Fiche signalétique de la BTK

Élément	Renseignement
Dénomination sociale	Banque Tuniso-Koweïtienne (BTK Bank)
Forme juridique	Société Anonyme (S.A.)
Date de création	25 février 1981
Capital social	200 000 000 DT (2 000 000 actions de 100 DT)
Siège social	10 bis, Avenue Mohamed V, 1001 Tunis
Secteur d'activité	Banque universelle
Actionnaire de référence	Groupe Elloumi (60 %)
Réseau	5 directions régionales, 6 succursales, 45 agences
Effectif (2024)	≈ 400 collaborateurs
Site web	www.btknet.com

en 2021. La banque conduit depuis un plan de transformation axé sur la consolidation financière et la digitalisation de ses services.

1.1.2 Structure et évolution du capital

Depuis sa création, le capital de la BTK a connu plusieurs évolutions, résumées dans le tableau 1.2. Au 30 septembre 2025, il est réparti entre le **Groupe Elloumi** (60 %, actionnaire de référence depuis 2021), l'**État tunisien** (20 %) et la **Kuwait Investment Authority** (20 %), cofondateur historique de l'institution.

TABLE 1.2 – Évolution de la structure du capital de la BTK

Année	Événement	Capital (MDT)	Actionnariat principal
1981	Création de la BTK (ex-BTKD)	100	État tunisien (50 %) – État du Koweït (50 %)
2008	Entrée du groupe BPCE International	100	BPCE (60 %) – État (20 %) – Koweït (20 %)
2017	Augmentation du capital	200	BPCE (60 %) – État (20 %) – Koweït (20 %)
2021	Acquisition par le Groupe Elloumi	200	Groupe Elloumi (60 %) – État (20 %) – KIA (20 %)

1.1.3 Activités et services

En tant que banque universelle, la BTK propose une gamme étendue de produits et de services : la **banque de détail** (comptes, cartes, crédits à la consommation et immobiliers, banque digitale via BTKNet), la **banque des entreprises** (crédits d'investissement et de fonctionnement, financement du commerce extérieur, gestion de trésorerie), les **opérations internationales et de marché**, ainsi que des **services spécialisés** (banque d'affaires, banque privée, bancassurance, intermédiation boursière).

1.2 Cadre et contexte du stage

Le stage s'est déroulé du **16 février 2026 au 16 juin 2026** au sein de la BTK, dans le cadre du projet de fin d'études. Le réseau d'agences de la banque génère un volume important d'informations relatives aux agences, aux collaborateurs, aux clients et aux objectifs commerciaux. La gestion de ces informations à l'aide d'outils bureautiques dispersés atteint rapidement ses limites en matière de cohérence, de traçabilité et de restitution.

1.3 Étude de l'existant

1.3.1 Description de l'existant

La gestion actuelle repose en grande partie sur des traitements manuels et des fichiers non centralisés. Les données relatives aux agences, aux employés et aux clients sont saisies et conservées de manière hétérogène, et les demandes de création ou de modification circulent par des canaux informels.

1.3.2 Critique de l'existant

Cette organisation présente plusieurs limites :

- dispersion et redondance des données, sources d'incohérences ;
- absence de circuit de validation formalisé pour les opérations sensibles ;
- faible traçabilité des actions et des décisions ;
- difficulté à produire des indicateurs consolidés pour le pilotage ;
- gestion des droits d'accès insuffisamment maîtrisée.

1.4 Problématique

Comment centraliser et fiabiliser la gestion du réseau d'agences bancaires, tout en encadrant les opérations sensibles par des circuits de validation, et comment exploiter les données ainsi structurées pour fournir aux responsables des indicateurs d'aide à la décision ?

1.5 Solution proposée

La solution consiste en une application web centralisée, structurée autour d’une base de données relationnelle Oracle, qui couvre la gestion des agences, des employés, des clients et des objectifs, et qui formalise les demandes de création et de modification au moyen de workflows de validation. Un volet décisionnel complète le dispositif : des vues analytiques alimentent Microsoft Power BI, et un tableau de bord embarqué restitue les indicateurs clés directement dans l’application.

1.6 Méthodologie de travail

La conduite du projet a suivi la méthodologie agile **Scrum**, fondée sur un développement itératif et incrémental. Le travail a été découpé en **sprints** successifs, chacun livrant un incrément fonctionnel : un *product backlog* regroupe les fonctionnalités à réaliser, décomposées en *sprint backlogs* ; chaque sprint se conclut par une revue et une rétrospective permettant d’ajuster la suite des travaux. Le tableau 1.3 et la figure 1.2 présentent le découpage en sprints et la planification correspondante.

TABLE 1.3 – Découpage du projet en sprints Scrum

Sprint	Objectif principal
Sprint 0	Cadrage, étude de l’existant et spécification des besoins
Sprint 1	Authentification et gestion des rôles
Sprint 2	Gestion des agences et des employés
Sprint 3	Gestion des clients et des objectifs
Sprint 4	Workflows de demandes (clients, employés, modifications)
Sprint 5	Volet décisionnel (vues BI et Power BI)
Sprint 6	Tests, consolidation et rédaction

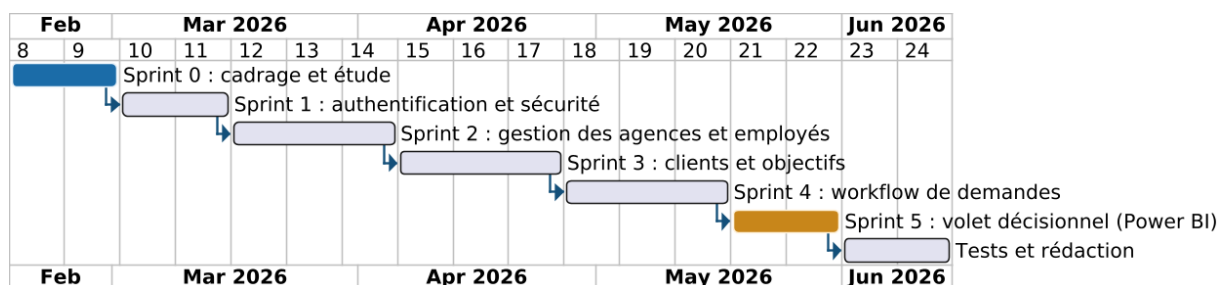


FIGURE 1.2 – Planification du projet (diagramme de Gantt des sprints)

Conclusion

Après avoir situé le projet, présenté l'organisme d'accueil et dégagé la problématique, le chapitre suivant formalise les besoins fonctionnels et non fonctionnels de la solution.

Chapitre 2

Analyse et spécification des besoins

Introduction

Ce chapitre identifie les acteurs du système, spécifie les besoins fonctionnels et non fonctionnels, puis les illustre au moyen du diagramme de cas d'utilisation et de la description de quelques cas représentatifs.

2.1 Identification des acteurs

Trois profils interagissent avec l'application, conformément aux rôles gérés par le module de sécurité (table SECURITY_USERS). Le tableau 2.1 en précise les responsabilités.

TABLE 2.1 – Identification des acteurs

Acteur	Responsabilités
Administrateur (ADMIN)	Gère les agences, les employés et les clients ; valide les demandes de création de clients et d'employés ; consulte le journal d'audit.
Directeur commercial (DIRECTEUR_COMMERCIAL)	Soumet les demandes de création (clients, employés) ; valide les demandes de modification des données des employés ; consulte les objectifs.
Utilisateur (USER)	Consulte les données autorisées et soumet des demandes de modification de ses propres informations.

2.2 Besoins fonctionnels

Le système doit permettre :

- l'authentification sécurisée et la gestion des rôles et des droits d'accès ;
- la gestion (CRUD) des agences, des employés et des clients ;
- le rattachement des employés et des clients à leur agence et à leur gestionnaire ;
- la consultation des objectifs commerciaux et le suivi de la production ;
- le **pointage (présence) des employés** : saisie de l'heure d'arrivée et de départ et détermination du statut (présent / retard / absent), avec saisie automatique (bouton) ou manuelle (administrateur) ;

- la soumission et la validation des demandes (clients, employés, modifications) selon le cycle EN_ATTENTE → EN_COURS → EFFECTUE/REFUSE;
- la notification des demandes en attente selon le rôle;
- la journalisation des actions (audit) et l’affichage de l’activité récente;
- la restitution d’indicateurs via un tableau de bord et via Power BI.

2.3 Besoins non fonctionnels

Le tableau 2.2 récapitule les besoins non fonctionnels.

TABLE 2.2 – Besoins non fonctionnels

Besoin	Description
Sécurité	Authentification, gestion des rôles et protection des opérations sensibles par des workflows de validation.
Fiabilité	Intégrité des données garantie par le SGBD et les transactions JTA.
Performance	Temps de réponse acceptable sur les volumes manipulés.
Ergonomie	Interface claire et cohérente, navigation par menu latéral.
Maintenabilité	Architecture en couches, code modulaire et faiblement couplé.
Portabilité	Application Jakarta EE standard, déployable sur serveur compatible.

2.4 Diagramme de cas d’utilisation global

La figure 2.1 présente le **diagramme de cas d’utilisation global**. Ce diagramme UML a pour objectif de représenter, d’un point de vue fonctionnel, les interactions entre les acteurs et les services offerts par le système.

On y distingue les trois acteurs identifiés et leurs cas d’utilisation respectifs. Le cas *S’authentifier* est commun à tous les acteurs ; il est relié aux autres cas par une relation «include» traduisant le fait que l’accès aux fonctionnalités exige une authentification préalable. L’**administrateur** concentre les cas de gestion (agences, employés, clients), la validation des demandes et la consultation du journal ; le **directeur commercial** soumet les demandes et valide les modifications ; l’**utilisateur** se limite à la consultation et à la soumission de demandes le concernant.

2.5 Description de quelques cas d’utilisation

2.5.1 Cas « S’authentifier »

L’utilisateur saisit son identifiant et son mot de passe. Le système vérifie ces informations auprès de la table de sécurité dédiée ; en cas de succès, une session est ouverte et l’utilisateur est redirigé vers le tableau de bord correspondant à son rôle ; sinon, un message d’erreur est affiché.

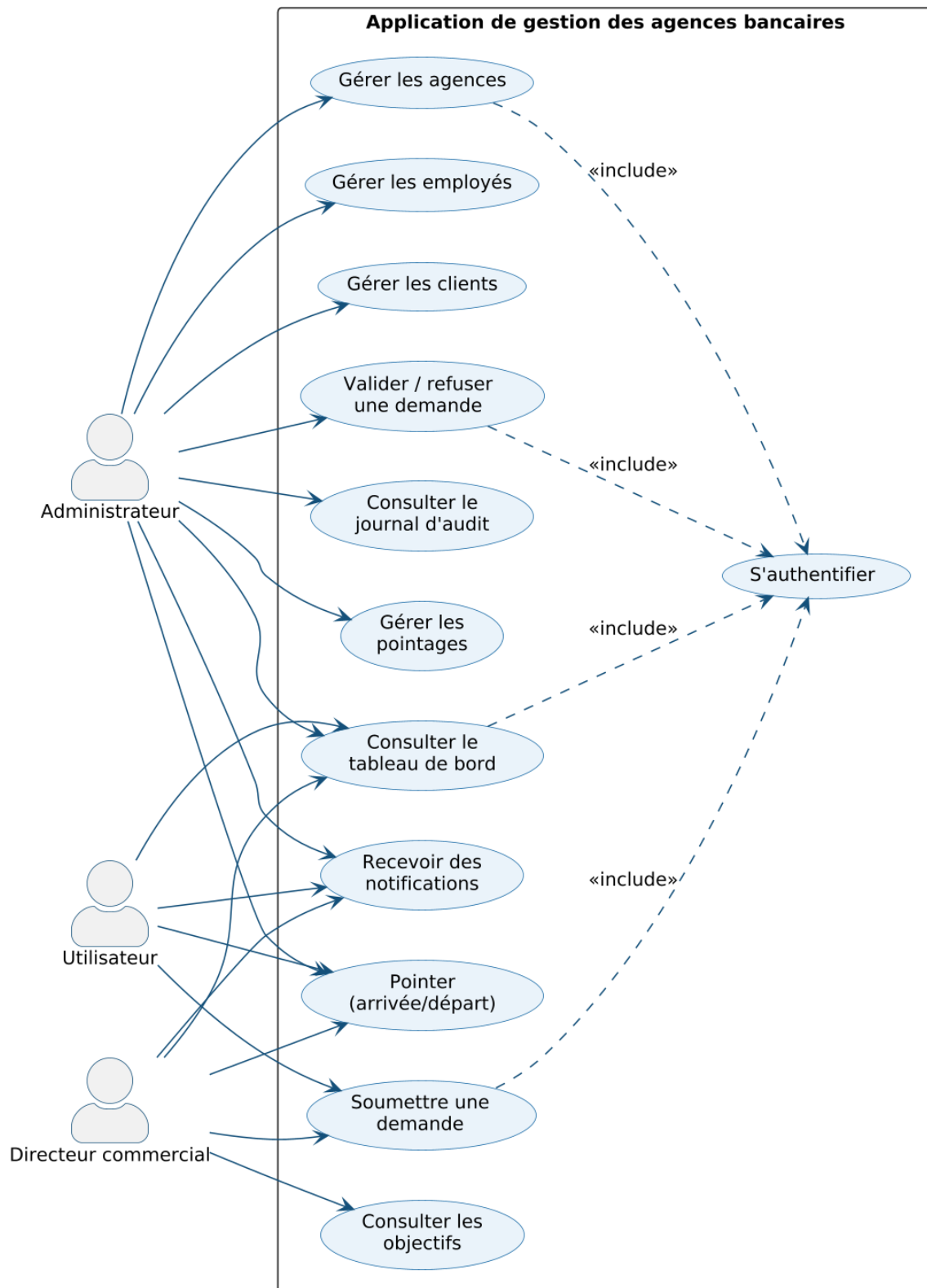


FIGURE 2.1 – Diagramme de cas d'utilisation global

2.5.2 Cas « Soumettre / valider une demande »

Le directeur commercial soumet une demande (création d'un client, d'un employé ou modification de données), enregistrée avec le statut EN_ATTENTE et notifiée à l'administrateur. Ce dernier peut l'approuver ou la refuser en y associant un commentaire ; le statut évolue alors vers EFFECTUE ou REFUSE, et l'action est journalisée.

Conclusion

Les besoins étant spécifiés et illustrés, le chapitre suivant présente la conception de l'application et de son volet décisionnel.

Chapitre 3

Conception

Introduction

Ce chapitre décrit la conception de la solution. Il présente successivement l'architecture générale, les architectures **logique** et **physique**, la conception détaillée (diagramme de classes, diagrammes de séquence et d'états-transitions), la conception de la base de données et celle du volet décisionnel.

3.1 Architecture générale

L'application adopte une architecture en couches reposant sur le patron **Modèle–Vue–Contrôleur** (MVC) et la plateforme Jakarta EE. La couche de présentation (JSP/JSTL) restitue les vues ; la couche de contrôle (Servlets) orchestre les requêtes ; la couche métier (EJB) encapsule la logique applicative et les transactions ; la couche de persistance (JPA/Hibernate) assure le mapping objet-relationnel vers la base Oracle.

3.2 Architecture logique

L'architecture logique, présentée à la figure 3.1, décompose le système en **couches** aux responsabilités distinctes et faiblement couplées.

Il s'agit d'un **diagramme de composants** dont l'objectif est de montrer l'organisation interne du logiciel et le flux des appels entre couches :

- **Couche présentation** : pages JSP/JSTL, feuilles de style CSS et tableau de bord Apex-Charts ; elle produit l'interface restituée au navigateur ;
- **Couche contrôle** : Servlets annotés @WebServlet et le filtre NotificationFilter ; elle reçoit les requêtes HTTP et invoque les services ;
- **Couche métier** : services EJB (@Singleton) portant la logique applicative et les transactions ;
- **Couche persistance** : entités JPA et Hibernate, qui dialoguent avec la base Oracle via l'EntityManager.

Microsoft Power BI se connecte directement à la base, en lecture, à travers les vues décisionnelles V_BI_.*.

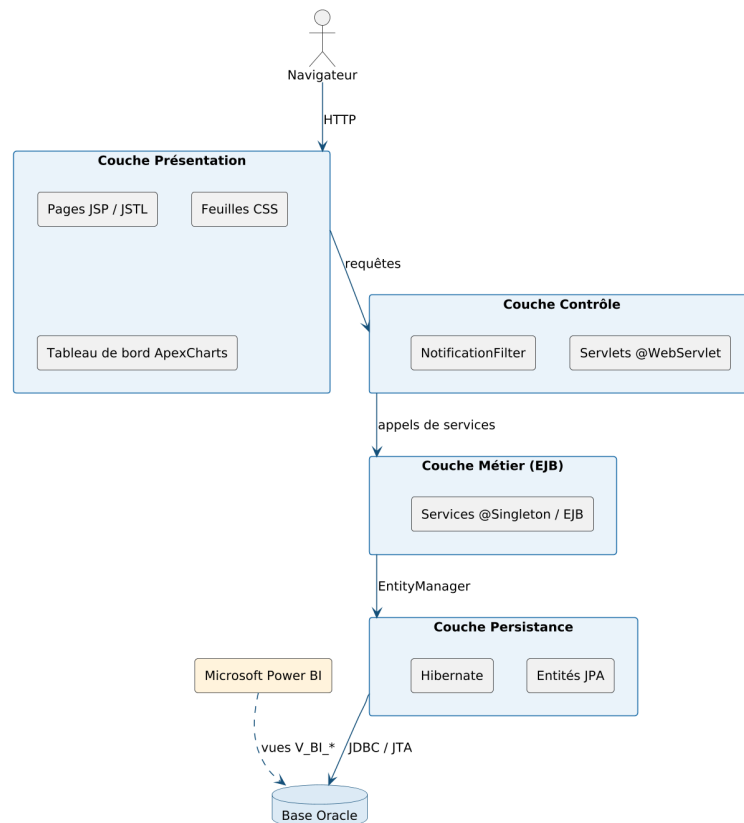


FIGURE 3.1 – Architecture logique en couches de l'application

3.3 Architecture physique

L'architecture physique, illustrée à la figure 3.2, représente le **déploiement** des composants sur les différents nœuds matériels et les liaisons réseau qui les relient.

Ce **diagramme de déploiement** UML décrit une organisation **3-tiers** :

- le **poste client** exécute un navigateur web qui dialogue avec le serveur applicatif en HTTP/HTTPS ;
- le **serveur d'applications** WildFly/JBoss héberge l'artefact déployé `gestion-agences.war` ; il accède à la base via une source de données JTA (OracleDS) en JDBC ;
- le **serveur de base de données** héberge l'instance Oracle (FREEPDB1) ;
- le **poste décisionnel** exécute Power BI Desktop, connecté à la base Oracle pour consommer les vues `V_BI_*`.

3.4 Conception détaillée

3.4.1 Diagramme de classes

La figure 3.3 présente le **diagramme de classes** du modèle métier. Son objectif est de représenter les entités persistantes du domaine, leurs attributs et les associations qui les relient.

L'entité **Utilisateur** occupe une position centrale. Les principales associations, fidèles au modèle réel, sont les suivantes :

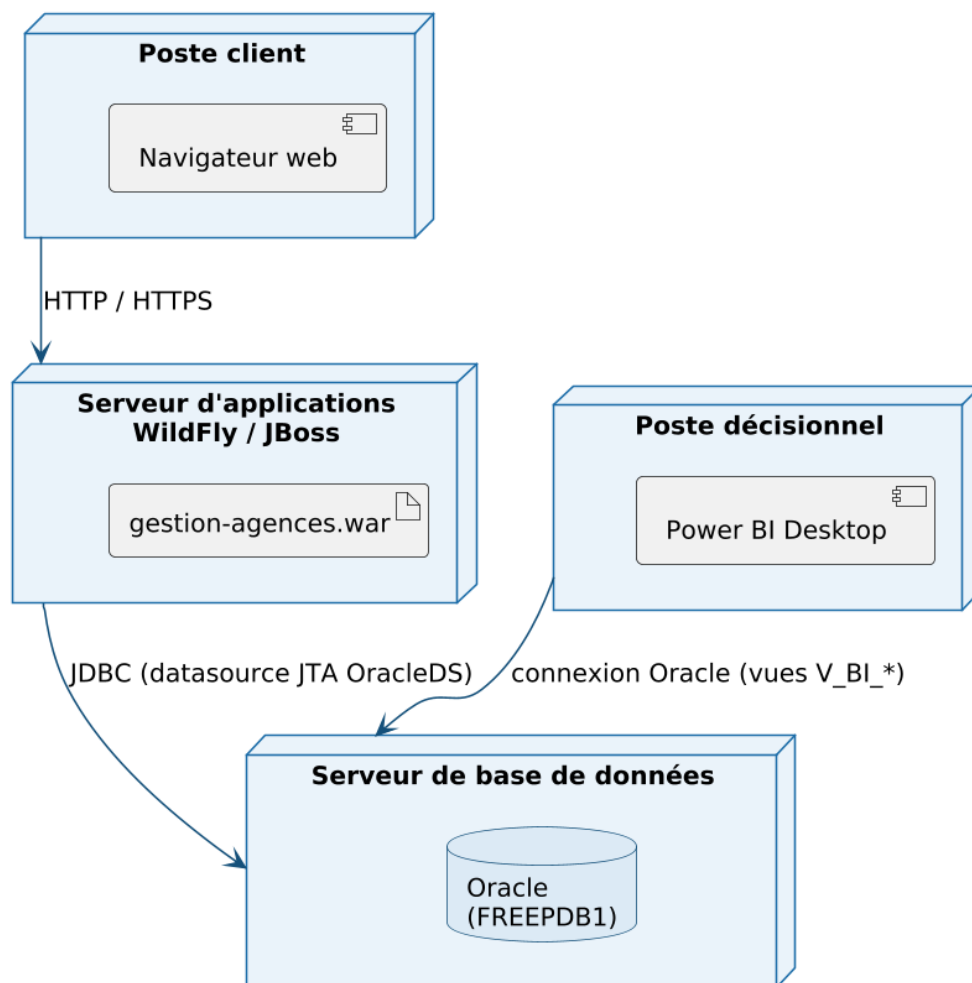


FIGURE 3.2 – Architecture physique (diagramme de déploiement 3-tiers)

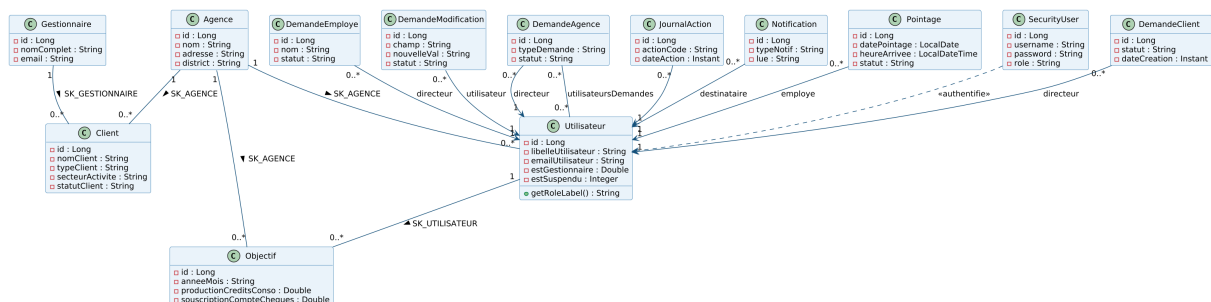


FIGURE 3.3 – Diagramme de classes du modèle métier

- une Agence regroupe plusieurs Utilisateur, Client et Objectif (cardinalité 1–0..*, via la clé étrangère SK_AGENCE);
- un Gestionnaire encadre plusieurs Client (SK_GESTIONNAIRE);
- chaque Objectif est rattaché à un Utilisateur (SK_UTILISATEUR);
- les entités de demande (DemandeClient, DemandeEmploye, DemandeModification, DemandeAgence), ainsi que JournalAction, Notification et Pointage, référencent un Utilisateur par une association @ManyToOne (0..* vers 1);
- DemandeAgence possède en outre une association @ManyToMany avec Utilisateur (utilisateursDem);
- SecurityUser est relié à Utilisateur par une **dépendance** «authenticatif», l'authentification établissant le lien entre identifiant de sécurité et utilisateur métier.

Le modèle ne comporte ni héritage ni composition entre entités : les liens sont exclusivement des associations, dont certaines sont matérialisées par des clés étrangères et d'autres par des relations JPA explicites. Ce choix reflète fidèlement le code du projet.

3.4.2 Diagramme de séquence : authentification

La figure 3.4 décrit, au moyen d'un **diagramme de séquence**, le déroulement dynamique du cas *S'authentifier*.

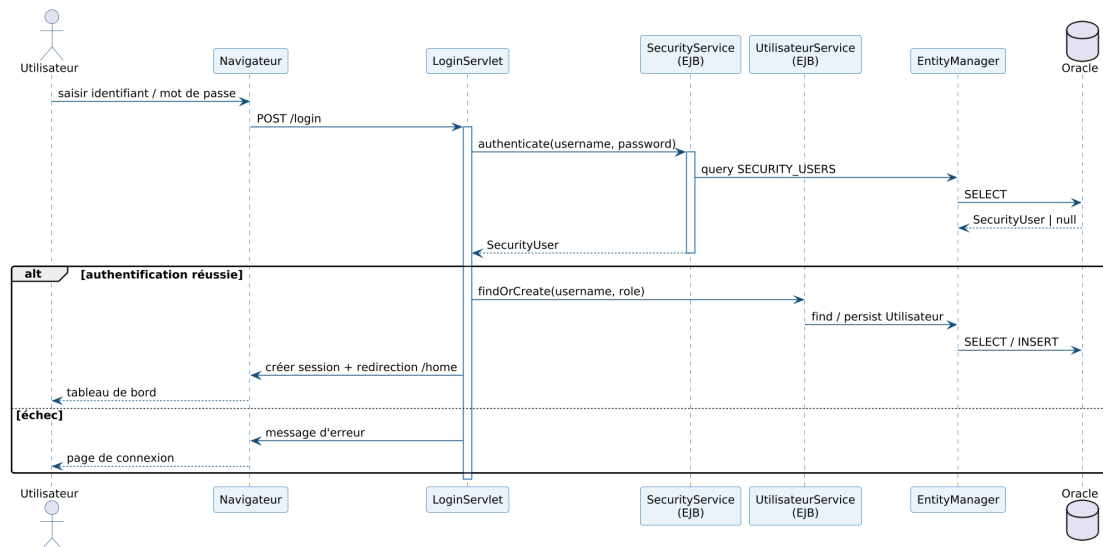


FIGURE 3.4 – Diagramme de séquence – Authentification

Le LoginServlet reçoit la requête POST /login et délègue la vérification au SecurityService, qui interroge la table SECURITY_USERS via l'EntityManager. Le fragment combiné alt distingue les deux issues : en cas de succès, le service UtilisateurService associe (ou crée) l'utilisateur métier, une session est ouverte et l'utilisateur est redirigé vers le tableau de bord ; en cas d'échec, un message d'erreur est renvoyé.

3.4.3 Diagramme de séquence : validation d'une demande

La figure 3.5 détaille le traitement d'une demande de modification par l'administrateur.

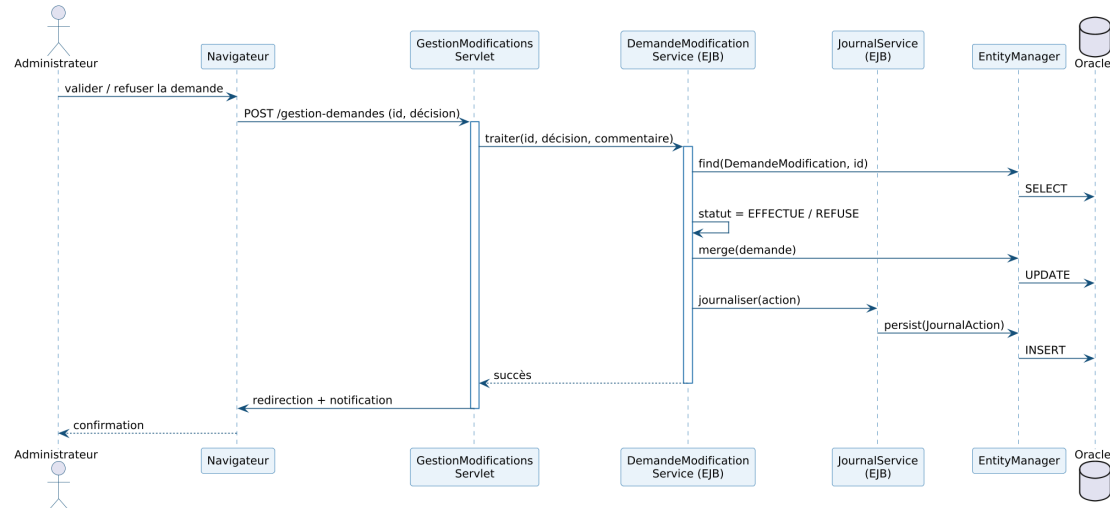


FIGURE 3.5 – Diagramme de séquence – Validation d’une demande

Le `GestionModificationsServlet` transmet la décision au service métier, qui charge la demande, met à jour son statut (EFFECTUE/REFUSE), persiste la modification puis journalise l’action via le `JournalService`. Ce diagramme met en évidence la collaboration entre les couches contrôle, métier et persistance.

3.4.4 Diagramme d’états-transitions : cycle de vie d’une demande

La figure 3.6 modélise, par un **diagramme d’états-transitions**, le cycle de vie d’une demande.

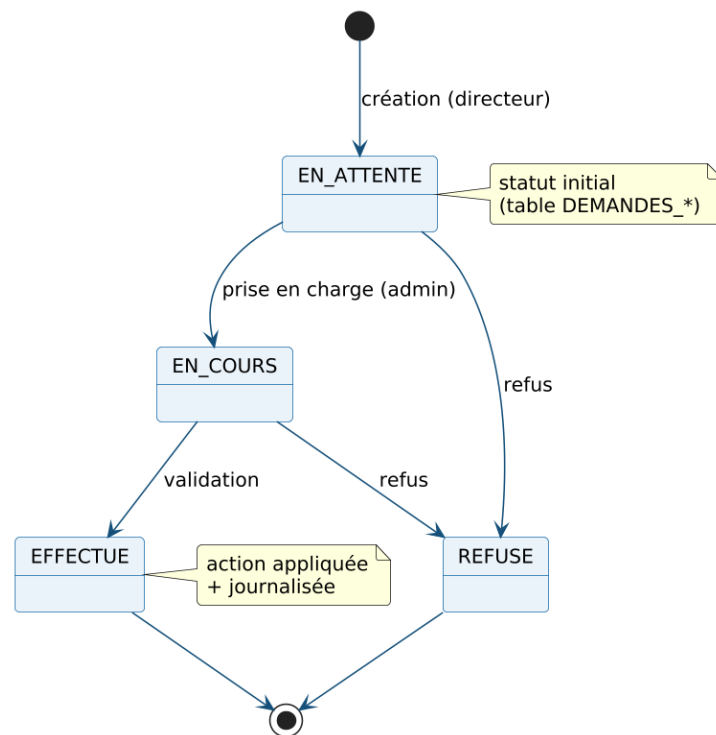


FIGURE 3.6 – Diagramme d’états-transitions – Cycle de vie d’une demande

Une demande naît à l'état EN_ATTENTE, passe en EN_COURS lors de sa prise en charge, puis atteint un état final EFFECTUE ou REFUSE. Ces états correspondent exactement aux valeurs autorisées par la contrainte de la table des demandes, ce qui garantit la cohérence entre le modèle de conception et la base de données.

3.5 Conception de la base de données

La figure 3.7 présente le **schéma relationnel** de la base.

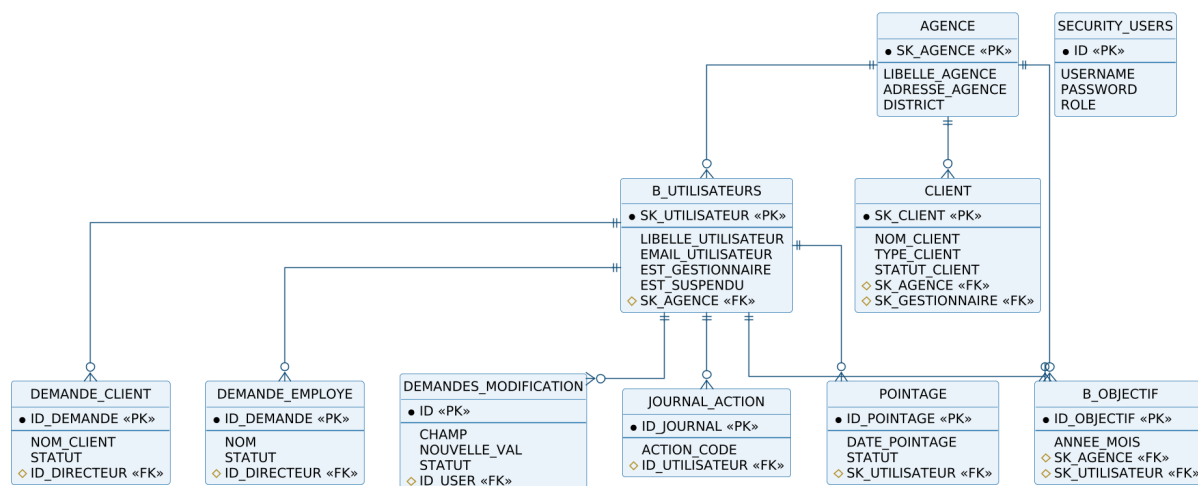


FIGURE 3.7 – Schéma relationnel de la base de données

Le schéma s'articule autour des tables AGENCE, B_UTILISATEURS, CLIENT et B_OBJECTIF, complétées par les tables de sécurité (SECURITY_USERS), de demandes (DEMANDE_CLIENT, DEMANDE_EMPLOYE, DEMANDES_MODIFICATION), de journalisation (JOURNAL_ACTION) et de pointage (POINTAGE). Les clés primaires (PK) et étrangères (FK) ainsi que les cardinalités (notation « patte d'oie ») y sont explicitées.

3.6 Conception du volet décisionnel

La couche décisionnelle transforme les données opérationnelles en informations de pilotage. Quatre vues analytiques jouent le rôle de couche sémantique selon une logique en **étoile** (figure 3.8) : autour de tables de faits (production, effectifs, clients, présence), des dimensions descriptives (agence, gestionnaire, période, type de client) permettent l'analyse multidimensionnelle.

Le tableau 3.1 récapitule les principaux indicateurs (KPI) retenus.

Conclusion

La conception étant établie — des architectures jusqu'au modèle décisionnel — le chapitre suivant présente la réalisation concrète de l'application.

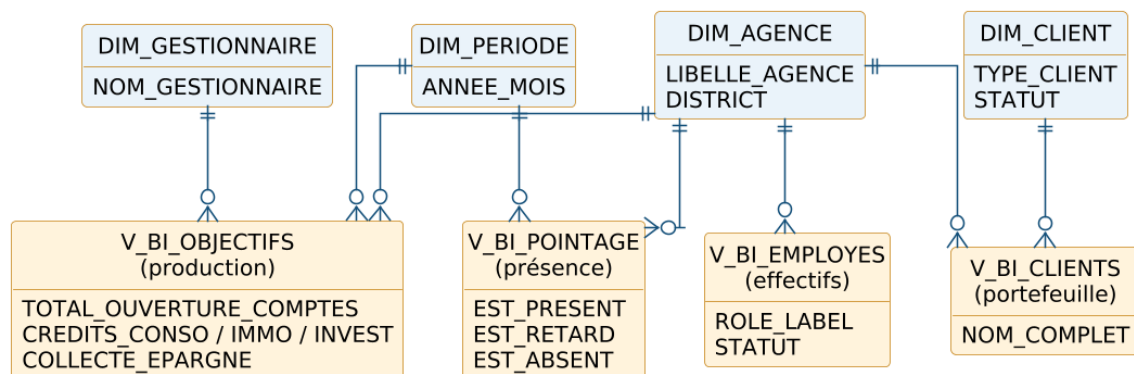


FIGURE 3.8 – Modèle décisionnel en étoile (vues V_BI_*)

TABLE 3.1 – Indicateurs de performance (KPI) du volet décisionnel

Indicateur	Description	Source
Effectifs par agence	Nombre d'employés par agence	V_BI_EMPLOYES
Répartition des rôles	Gestionnaires / conseillers / hors commercial	V_BI_EMPLOYES
Portefeuille clients	Clients par type et par agence	V_BI_CLIENTS
Ouvertures de comptes	Comptes chèques + épargne + courants	V_BI_OBJECTIFS
Production de crédits	Conso, immobilier, investissement	V_BI_OBJECTIFS
Collecte d'épargne	Épargne additionnelle collectée	V_BI_OBJECTIFS
Taux de présence	Présents, retards et absences par agence et période	V_BI_POINTAGE

Chapitre 4

Réalisation

Introduction

Ce chapitre présente l'environnement de travail, les choix technologiques, puis les principales interfaces de l'application et la réalisation du volet décisionnel.

4.1 Environnement de travail

4.1.1 Environnement matériel

Le développement a été réalisé sur un poste de travail standard [*à compléter : processeur, mémoire vive, système d'exploitation*].

4.1.2 Environnement logiciel

Le tableau 4.1 récapitule les principaux outils et technologies employés.

TABLE 4.1 – Environnement logiciel

Outil / Technologie	Rôle
JDK 21	Plateforme d'exécution et de compilation Java
Jakarta EE 10	Spécifications d'entreprise (Servlets, JSP, EJB, JPA)
Hibernate	Implémentation JPA (mapping objet-relationnel)
Oracle Database	Système de gestion de base de données
WildFly / JBoss	Serveur d'applications (déploiement du WAR)
Apache Maven	Gestion des dépendances et automatisation du build
IntelliJ IDEA	Environnement de développement intégré
ApexCharts	Graphiques du tableau de bord embarqué
Microsoft Power BI	Restitution et reporting décisionnel
Git	Gestion de versions du code source

4.2 Choix technologiques

Le choix de **Jakarta EE 10** se justifie par sa robustesse, sa standardisation et son adéquation aux applications d'entreprise : injection de dépendances, gestion transactionnelle déclarative (JTA), persistance via JPA et sécurité intégrée. La base **Oracle** a été retenue pour sa fiabilité et sa large adoption dans le secteur bancaire. Pour la restitution décisionnelle, **Power BI** offre une connexion directe aux vues Oracle et des capacités de visualisation avancées, complétées par un tableau de bord embarqué (**ApexCharts**) pour une consultation immédiate au sein de l'application.

4.3 Architecture de déploiement

L'application est empaquetée sous forme d'archive WAR (gestion-agences) et déployée sur WildFly sous le contexte /gestion-agences. L'accès à la base s'effectue via une source de données JTA (OracleDS) gérée par le conteneur, conformément à l'architecture physique du chapitre 3.2.

4.4 Présentation des interfaces

Les figures suivantes présentent les principales interfaces de l'application. *Les captures d'écran réelles seront insérées à cet emplacement.*

4.4.1 Page d'authentification

La figure 4.1 présente la page de connexion.

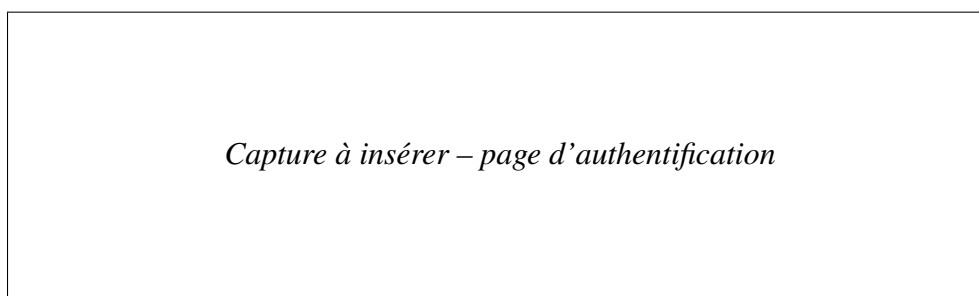


FIGURE 4.1 – Interface d'authentification

4.4.2 Tableau de bord

La figure 4.2 présente le tableau de bord et ses graphiques ApexCharts (effectifs par agence, répartition des rôles).

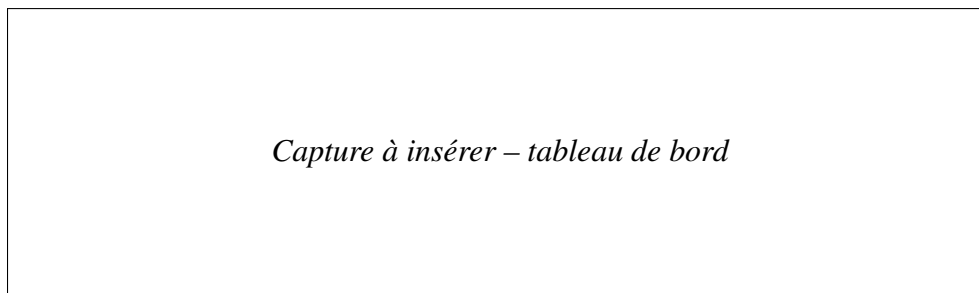


FIGURE 4.2 – Tableau de bord de l'application

4.4.3 Modules de gestion et de demandes

La figure 4.3 illustre les modules de gestion (agences, employés, clients) et le traitement des demandes.

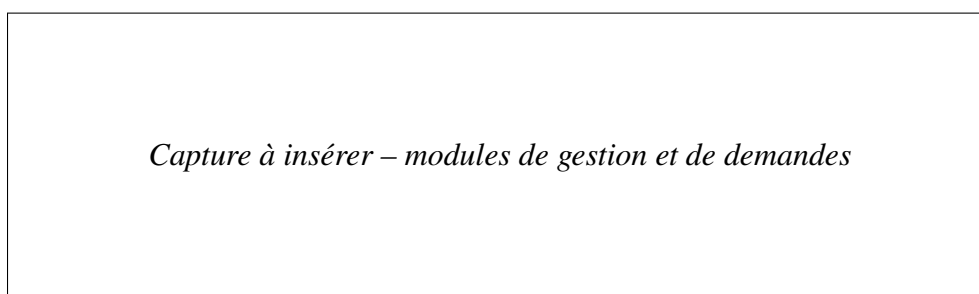


FIGURE 4.3 – Modules de gestion et de validation des demandes

4.5 Réalisation du volet décisionnel

Les vues analytiques (V_BI_EMPLOYES, V_BI_CLIENTS, V_BI_OBJECTIFS) ont été créées dans la base Oracle puis connectées à Power BI, où des rapports interactifs analysent les effectifs, le portefeuille clients et la production par agence et par période (figure 4.4).

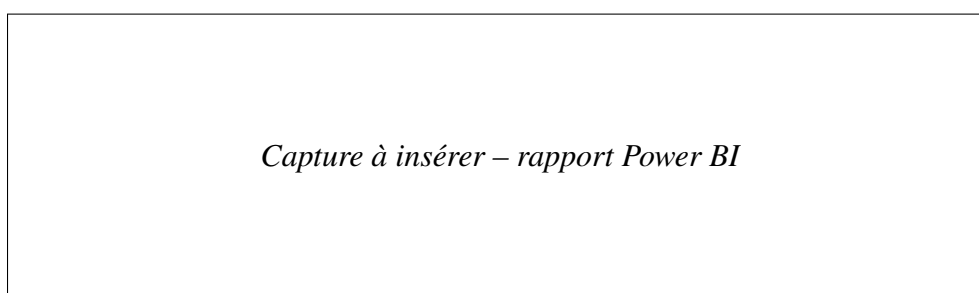


FIGURE 4.4 – Rapport Power BI – analyse des effectifs et des objectifs

4.6 Module de pointage

Le module de pointage assure le suivi de la présence des employés. Chaque employé enregistre son arrivée et son départ via un bouton « Pointer » (l'heure est relevée sur l'heure système); le statut est déterminé automatiquement (PRESENT, ou RETARD au-delà de 9h00). L'administrateur peut en outre saisir ou corriger manuellement un pointage (PRESENT, RETARD ou ABSENT). Techniquement, le module repose sur l'entité Pointage, le service PointageService (EJB) et le servlet PointageServlet (/pointage); il alimente la vue décisionnelle V_BI_POINTAGE (taux de présence, retards, absences par agence et période). La figure 4.5 en présente l'interface.

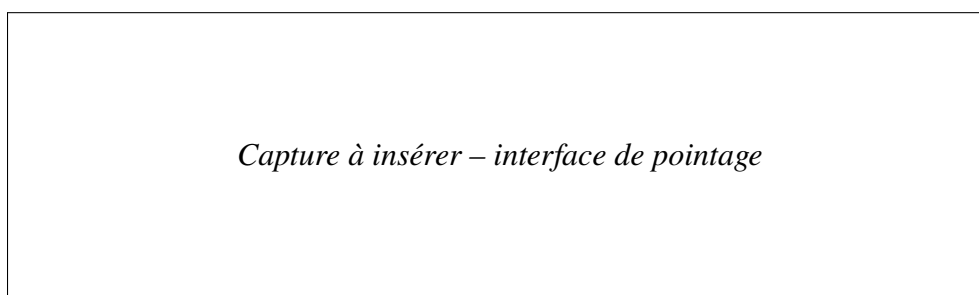


FIGURE 4.5 – Interface du module de pointage

4.7 Sécurité et gestion des rôles

L'accès à l'application est contrôlé par le SecurityService, qui authentifie les utilisateurs à partir de la table dédiée et associe à chacun un rôle (ADMIN, DIRECTEUR_COMMERCIAL, USER). Ce rôle conditionne l'accès aux fonctionnalités. Les opérations sensibles sont encadrées par les workflows de validation, et chaque action significative est journalisée à des fins d'audit.

Conclusion

La réalisation a permis d'aboutir à une application fonctionnelle couvrant les besoins exprimés, complétée par un dispositif décisionnel opérationnel.

Conclusion générale et perspectives

Ce projet de fin d'études a permis de concevoir et de réaliser, au sein de la **BTK – Banque Tuniso-Koweïtienne**, une application web de gestion des agences bancaires adossée à un volet décisionnel. La solution centralise la gestion des agences, des employés, des clients et des objectifs, formalise les opérations sensibles au moyen de circuits de validation, et restitue aux responsables des indicateurs d'aide à la décision via un tableau de bord embarqué et des rapports Power BI.

Sur le plan technique, ce travail a été l'occasion de mettre en pratique les compétences acquises durant la formation : modélisation UML, développement Jakarta EE (Servlets, JSP, EJB, JPA), conception de bases de données Oracle et mise en œuvre d'une chaîne décisionnelle (vues analytiques et reporting Power BI), au croisement du développement logiciel et de la Business Intelligence.

Plusieurs perspectives d'évolution peuvent être envisagées :

- le chiffrement (hachage) des mots de passe et le renforcement de la sécurité ;
- l'automatisation de l'alimentation d'un entrepôt de données (ETL) pour historiser les indicateurs ;
- l'enrichissement par des analyses prédictives (prévision des objectifs) ;
- l'exposition d'une API REST pour l'interopérabilité avec d'autres systèmes ;
- le développement d'une application mobile de consultation des indicateurs.

Nétographie

Annexe A

Annexes

A.1 Scripts SQL d'initialisation

Les scripts de création des tables et des séquences (sécurité, demandes, modifications) ainsi que les définitions des vues décisionnelles (V_BI_EMPLOYES, V_BI_CLIENTS, V_BI_OBJECTIFS) figurent dans le dépôt du projet, sous le répertoire `sql/`.

A.2 Captures complémentaires

Des captures d'écran complémentaires de l'application et des rapports Power BI pourront être insérées dans cette annexe.

A.3 Dépôt du code source

Le code source complet du projet est disponible sur le dépôt GitHub associé au rapport.